



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Análisis de temas en Trabajos
Fin de Grado**



Presentado por Zeldan Javier Campos Cordero
en Universidad de Burgos — 31 de diciembre
de 2025

Tutor: Carlos López Nozal



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Carlos López Nozal, profesor del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos.

Expone:

Que el alumno D. Zeldan Javier Campos Cordero, con DNI 55462889E, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Análisis de temas en Trabajos Fin de Grado.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 31 de diciembre de 2025

Vº. Bº. del Tutor:

D. Carlos López Nozal

Resumen

TFG topics es una aplicación que pretende realizar análisis temáticos cualitativos de los textos contenidos en las memorias de los trabajos finales de grado (TFG). Su objetivo es desarrollar una herramienta que permita a los docentes y estudiantes explorar e identificar tendencias en el contenido académico de dichos trabajos.

Para ello, se parte de un conjunto de datos creado con las memorias de TFG en formato LaTeX disponibles públicamente en GitHub. Sobre estos textos se lleva a cabo el procesamiento y modelado de temas utilizando diversos algoritmos como: **Latent Dirichlet Allocation (LDA)** [27], **Top2Vec** [21], **BERTopic** [35] y **FASTopic** [44] [44].

Este trabajo se centra en la implementación, comparación y evaluación de diferentes algoritmos de modelado de temas (**Topic Modeling**), con el fin de construir una herramienta que ayude a la gestión del conocimiento académico en el ámbito universitario.

Descriptores

Análisis temático, Topic Modeling, LDA, BERTopic, Top2Vec, FASTopic, TFG, Análisis de Datos, Procesamiento del Lenguaje Natural, Ingeniería Informática.

Abstract

TFG Topic is an application designed to perform qualitative thematic analysis of the texts contained in Bachelor's Thesis (TFG) reports. Its main goal is to develop a tool that enables teachers and students to explore and identify trends within the academic content of these works.

To achieve this, a dataset was built from TFG reports written in LaTeX format and publicly available on GitHub. These texts are processed and analyzed using various topic modeling algorithms, such as **Latent Dirichlet Allocation (LDA)**, **Top2Vec**, **BERTopic**, and **FASTopic**.

This work focuses on the implementation, comparison, and evaluation of different **Topic Modeling** algorithms, with the aim of building a tool that supports academic knowledge management within the university context.

Keywords

Topic Modeling, LDA, BERTopic, Top2Vec, FASTopic, TFG, Data Analysis, PNL, Computer Science.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
2.3. Objetivos personales	4
3. Conceptos teóricos	5
3.1. Procesamiento de lenguaje natural	5
3.2. Preprocesamiento del textos	6
3.3. Representación del texto	9
3.4. Modelado de temas	10
3.5. Evaluación de modelos	14
3.6. Hiperparametrización de Modelos	15
3.7. Comparación de modelos	18
4. Técnicas y herramientas	19
4.1. Metodologías de desarrollo	19
4.2. Herramientas de gestión de proyectos	20
4.3. Herramientas de desarrollo	20
4.4. Documentación	22

5. Aspectos relevantes del desarrollo del proyecto	25
5.1. Arquitectura y diseño del software	25
6. Trabajos relacionados	27
7. Conclusiones y Líneas de trabajo futuras	29
Bibliografía	31

Índice de figuras

Índice de tablas

3.1. Comparativa de las características de los algoritmos de modelado de temas.	18
--	----

1. Introducción

En el ámbito universitario anualmente se generan gran cantidad de documentos académicos que proporcionan una valiosa fuente de información. Especialmente las memorias de los **Trabajos de Fin de grado (TFG)** que poseen un alto valor y que forman parte fundamental del proceso formativo del estudiante, ya que permiten aplicar los conocimientos adquiridos durante la realización de los estudios de grado [38]. Estas memorias contienen información sobre tendencias tecnológicas actuales, herramientas y investigaciones realizadas sobre distintas áreas de la informática. Sin embargo, resulta muy laborioso ubicar y extraer esta información ya que encuentra almacenada de manera dispersa en distintos repositorios y formatos.

Este trabajo tiene como finalidad el desarrollo de una aplicación para el análisis cualitativo de los textos contenidos en las memorias de TFG del grado en ingeniería Informática de la Universidad de Brugos. Esta herramienta permite el análisis, consulta, facilita la exploración, calculo de tendencias y comparación de trabajos a lo largo del tiempo partiendo de estos contenidos textuales. El conjunto de datos analizado se encuentra en formato $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ y se encuentra disponible en plataformas como GitHub, a estos datos se le aplicaran técnicas de modelado de temas utilizando distintos algoritmos de última generación, entre ellos **Latent Dirichlet Allocation (LDA)** [27], **Top2Vec** [31], **BERTopic** [35] y **FASTopic** [44].

La aplicación resultante servirá como ayuda a la comunidad docente, investigadora y estudiantil para la exploración de la información académica. Además, se busca poner en práctica los conocimientos adquiridos y competencias durante la titulación.

Este proyecto se fundamenta en el uso de técnicas de modelado de temas. El modelado de temas es una tarea de aprendizaje no supervisado donde

se identifica la estructura temática de un grupo de documentos [24]. Al utilizar estas técnicas es posible encontrar temas relevantes para organizar, buscar o comprender grandes cantidades de datos en formato de texto. Estos modelos se basan en el supuesto de cada documento puede explicarse como una mezcla de temas, donde cada tema es un grupo de términos que poseen una distribución de potabilidades.

Esta memoria se estructura de la siguiente manera: **Introducción**, se realiza una descripción inicial de el trabajo y su estructura; **Objetivos**, descripción de los objetivos generales y específicos que se pretenden alcanzar con la elaboración de este proyecto; **Conceptos teóricos**, en esta sección se presenta el marco teórico y conceptual que fundamenta el modelado de temas; **Técnicas y herramientas**, se describen las técnicas y herramientas utilizadas para la elaboración de la aplicación; **Aspectos relevantes del desarrollo del proyecto**, se presentan los desafíos superados durante la realización del proyecto, la metodología seguida y el diseño del sistema; **Trabajos relacionados**, se exponen otros proyectos y aplicaciones similares y se realizar una comparación con el proyecto actual; **Conclusiones y líneas de trabajo futura**, los resultados obtenidos y su análisis comparativo y se detallan las conclusiones y las posibles líneas de trabajo futuro.

2. Objetivos del proyecto

El presente Trabajo Fin de Grado tiene como objetivo principal el desarrollo de una aplicación capaz de realizar análisis temáticos cualitativos sobre los textos contenidos en las memorias de los Trabajos Fin de Grado (TFG) en Ingeniería Informática. Las memorias, *issues*, *commits*, demás documentos descriptivos del TFG y código fuente se encuentran disponibles en repositorios de software almacenados en la plataforma GitHub.

Este proyecto busca aplicar técnicas de procesamiento del lenguaje natural y modelado de temas para facilitar la identificación de tendencias, temas recurrentes y áreas de investigación dentro del ámbito académico.

2.1. Objetivo general

Diseñar e implementar una aplicación que permita realizar análisis temáticos automáticos sobre la documentación disponible en el repositorio del TFG, como *readmes*, *issues*, *commits* y memorias en formato $\text{\LaTeX}$, utilizando para esto diversos algoritmos de modelado de temas y evaluando su rendimiento en términos de coherencia y eficiencia.

2.2. Objetivos específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

- **Recopilar y preparar los datos:** Crear un conjunto de datos utilizando la información de los TFG disponibles públicamente en GitHub, realizando el preprocesamiento necesario de los textos.

- **Implementar diferentes algoritmos de modelado de temas:** Investigar, aplicar y comparar métodos como **Latent Dirichlet Allocation (LDA)**, **Top2Vec**, **BERTopic** y **FASTopic**.
- **Evaluar el rendimiento de los modelos:** Medir y comparar la calidad de los temas generados utilizando métricas de coherencia temática.
- **Desarrollar una interfaz visual:** Diseñar una aplicación que permita la visualización de los resultados obtenidos.
- **Fomentar la gestión del conocimiento académico:** Promover el acceso y la reutilización del conocimiento generado por estudiantes en cursos anteriores, apoyando la investigación y la docencia.
- **Diseño de arquitectura:** Diseñar e implementar una arquitectura de software modular que separe el motor de análisis de la capa de servicio, demostrando principios de Ingeniería de Software.
- **Gestión:** Utilizar herramientas y metodologías ágiles para planificar y implementar el proyecto. Utilizando Zube.io y GitHub.

2.3. Objetivos personales

- Crear una herramienta que sea de utilidad para la comunidad universitaria.
- Aplicar conocimientos adquiridos durante el grado, especialmente en el ámbito de Desarrollo Avanzado de Sistemas Software y Minería de Datos.
- Consolidar conocimientos en las siguientes tecnologías: Python, git, docker.
- Mejorar habilidades de diseño de software, aplicación de buenas practicas en desarrollo, testing de software y documentación.

3. Conceptos teóricos

A continuación se exponen conceptos teóricos que sientan las bases del proyecto, cubriendo desde el pre-procesamiento de texto hasta los algoritmos avanzados de modelado.

3.1. Procesamiento de lenguaje natural

El procesamiento de lenguaje natural (PLN, del inglés *Natural Language Processing*, NLP) es un campo de la Inteligencia Artificial (IA, del inglés *Artificial Intelligence*, AI) que se ocupa del análisis y la interpretación de textos que se encuentran en lenguaje humano. Utilizando estas técnicas los computadores pueden dar sentido a los datos contenidos en un conjunto de textos y así adquirir su conocimiento implícito [23].

NLP usa gran variedad de técnicas y modelos para analizar diferentes aspectos de los textos como la formación de palabras, el léxico, la sintaxis entre otros, con esto se pretende resolver problemas específicos relacionados con el lenguaje.

Entre las técnicas más usadas en NLP se encuentran la tokenización, lematización, eliminación de palabras vacías (del inglés *stopwords*), análisis sintáctico y extracción de información. En el contexto de este trabajo, las técnicas de NLP se utilizan para preparar los textos de las memorias de TFG, transformándolos en datos estructurados que puedan ser analizados mediante algoritmos de modelado de temas.

3.2. Preprocesamiento del textos

Antes de iniciar cualquier análisis los textos deben someterse a una serie de procesos para garantizar su limpieza, normalización y reducir dimensión del vocabulario, con el fin de garantizar la calidad de los análisis resultantes.

Tokenización

a La tokenización es un proceso fundamental dentro de el NLP y la AI, pues permite dividir el texto en unidades mínimas denominadas tokens, que pueden ser palabras, subpalabras, caracteres o unidades semánticas mas complejas [37]. Este proceso constituye la base para la mayoría de técnicas posteriores de análisis textual, ya que permite transformar cadenas de texto sin estructura en secuencias manejables por algoritmos computacionales.

La tokenización presenta una serie de problemas que son específicos de cada idioma. Por lo tanto, se requiere conocer el idioma del documento. La identificación del idioma mediante clasificadores que usan subsecuencias cortas de caracteres como características resulta muy efectiva, ya que la mayoría de los idiomas tienen patrones distintivos [11].

Tipos de Tokenización

Existen varios métodos comunes para realizar esta segmentación

Tokenización de palabras El texto se divide en palabras individuales basándose en un delimitador, típicamente el espacio. Esta representación puede fallar con palabras compuestas o en idiomas sin espacios entre palabras (como el chino) [33].

Tokenización de caracteres El texto se divide en caracteres individuales. Este método es útil para lenguajes con alfabetos complejos y permite al modelo manejar un rango más amplio de entradas, como palabras desconocidas o errores tipográficos [28, 33].

Tokenización de subpalabras El texto se divide en subpalabras o fragmentos frecuentes de caracteres. Este método reduce la dimensionalidad y maneja eficazmente palabras raras y desconocidas [28, 33].

Tokenización de espaciado blanco Divide el texto basándose únicamente en los espacios, lo que resulta en una gestión incorrecta de puntuaciones o errores gramaticales [28].

En el contexto del análisis temático de TFG, la tokenización cumple varias funciones clave:

- Estandarizar los textos, permitiendo compararlos entre sí.
- Reduce ruido, al separar puntuación, símbolos o caracteres no relevantes.
- Facilita operaciones posteriores, como lematización, extracción de n-grams o cálculo de frecuencias.
- Mejora la precisión del modelado de temas, ya que una buena segmentación permite que los modelos identifiquen patrones semánticos coherentes.

En particular para este trabajo la tokenización presenta varios problemas o desafíos:

Tratamiento de compuestos palabras como “aprendizaje-servicio” deben decidirse como uno o dos tokens.

Abreviaturas universitarias “TFG”, “EPS”, “ECTS”, etc., que pueden requerir reglas específicas.

Símbolos técnicos o matemáticos comunes en textos de informática.

Preservación del contexto eliminar puntuación puede afectar el significado y pérdida de contexto en algunos casos.

La tokenización es un proceso esencial y su correcta implementación afecta directamente la calidad del análisis temático y de los modelos posteriores, desde la extracción de palabras clave hasta el modelado de temas.

Lematización

La lematización es una técnica clave dentro del NLP que tiene como principal objetivo transformar o reducir las palabras a su forma canónica o lema, es decir, la forma bajo la cual se encontrarían en un diccionario. En muchas lenguas, el infinitivo se utiliza como lema para el verbo; por ejemplo, en español, dormir es el lema para la forma duermes [37].

El lema (*lemma*) se define como la forma de citación o forma base de una palabra. El concepto del lema es relevante para comprender la estructura del

significado. Un lema puede ser polisémico, es decir, puede tener múltiples sentidos. Cada aspecto individual del significado de un lema se denomina sentido de palabra (*word sense*). Por ejemplo, el lema ratón tiene al menos dos sentidos distintos: El roedor ó el dispositivo de control de cursor [37].

La lematización, al reducir las palabras a su forma base, asegura que un sistema de PLN trate variantes flexionales (como singular, plural o diferentes tiempos verbales) como la misma unidad, facilitando la generalización sobre el significado fundamental de la palabra.

Stopwords

Las palabras vacías (del inglés *stopwords*) es un termino que hace referencia a las palabras que deben ser eliminadas en la fase de preprocesamiento de texto para garantizar la precisión y eficiencia de las tareas de NLP. Estas palabras aparecen frecuentemente en muchos documentos, pero transmiten poca o nula información sobre la sección del texto al que pertenecen [41].

Al eliminar estas palabras que aportan baja información, se incrementa la significación estadística de los términos que si son relevantes y son verdaderamente importantes para la tarea. Ejemplos específicos de estas palabras son: cada, sobre, tal, el, la, entre otros [41].

Históricamente, se han realizado esfuerzos para identificar *stopwords* a partir de fuentes de conocimiento genéricas, como *Brown Corpus* o *20 newsgroup corpus*. Esto ha provocado la creación de listas genéricas que se utilizan en aplicaciones de NLP. Una de las listas más utilizadas es la distribuida con el paquete de Python NLTK (del inglés, *Natural Language Tool Kit*) [41].

Sin embargo, el lenguaje técnico utilizado en textos de ingeniería o tecnología se distingue del lenguaje común. El lenguaje técnico contiene su propio argot y palabras altamente frecuentes y poco informativas que no están presentes en las listas genéricas. No existe una lista estándar de *stopwords* para aplicaciones de procesamiento de lenguaje técnico. La adopción simple de listas genéricas deja muchos términos repetitivos y poco informativos específicos del dominio en los datos. Es por esto que el proceso de identificación de *stopwords* incluye una etapa posterior de evaluación humana por parte de expertos para confirmar que el término no contiene información sobre ingeniería y tecnología [41].

3.3. Representación del texto

Bolsa de palabras

La bolsa de palabras (del inglés *Bag of Words*, BOW) es una de las técnicas de representación de texto utilizada por distintas aplicaciones de NLP. Esta representación se basa en el conteo de palabras, ignorando el orden y la estructura gramatical del texto [36].

El método BOW implica la construcción de un vocabulario compuesto por todas las palabras únicas presentes en los documentos analizados y la definición de una métrica concreta para determinar el peso de cada término. Las métricas más empleadas para este fin son la frecuencia de términos (del inglés *term frequency*, TF), y la frecuencia inversa de documentos (del inglés *Term frequency-inverse document frequency*, TF-IDF) [36].

Una de las principales limitaciones de BOW es la pérdida de información contextual de los términos dentro de los documentos. Otro inconveniente de BOW es que no captura el significado de las palabras y trata todas las palabras como entidades independientes dentro del documento. Además, la vectorización mediante BW genera vectores dispersos de alta dimensión cuando se trabaja con corpus extensos, lo que incrementa la complejidad computacional y los requisitos de memoria. BOW no es capaz de gestionar los sinónimos, tratando palabras con un mismo significado como si estuvieran relacionadas [36].

Representaciones vectoriales

Debido a las limitaciones encontrados en otras formas de representación de texto como BOW, han surgido técnicas que buscan solucionar estos problemas, entre las cuales podemos encontrar a los *embeddings*. Los *embeddings* son una técnica de procesamiento de lenguaje natural que convierte el lenguaje humano en vectores matemáticos. Estos vectores son una representación del significado subyacente de las palabras, lo que permite que las computadoras procesen el lenguaje de manera más efectiva [29].

Los *embeddings* intentan solucionar el problema de la pérdida de contexto y semántica de BOW considerando el orden y la secuencia de aparición de cada término. Estos modelos se construyen sobre redes neuronales artificiales (del inglés *artificial neural networks*, ANN) y se entrenan con grandes cantidades de datos para capturar el significado contextual de una unidad léxica en función de las palabras que la rodean, y viceversa. Cada palabra en el espacio de *embeddings* se representa como un vector de números reales

con una dimensión predefinida. En un espacio de vectores de palabras, es posible identificar las palabras más similares a un término concreto del vocabulario utilizando métodos como la similitud del coseno o la distancia euclidiana. Estas técnicas permiten agrupar palabras relacionadas, clasificar textos, categorizar documentos y descubrir temas o tópicos latentes dentro del texto [36].

3.4. Modelado de temas

El modelado de temas (del inglés *Topic Modeling*) es una técnica muy utilizada con la cual podemos encontrar temas subyacentes dentro de un grupo de documentos que deseamos analizar. Esta sigue un enfoque estadístico y proporciona una forma de representar textos de una manera estructurada, lo que la hace ideal para analizar grandes conjuntos de datos textuales [26].

Dependiendo del algoritmo utilizado los términos se pueden obtener utilizando distintos enfoques como la bolsa de palabras BOW, frecuencia de términos, frecuencia inversa de términos o basado en clases [39].

Podemos destacar la importancia del modelado de temas en los siguientes aspectos [26]:

Organización de la información permite categorizar los documentos en temas lo cual hace mas fácil el proceso de búsqueda.

Resumen permite capturar la esencia de la temática del documento.

Recomendación facilita realizar recomendaciones personalizadas para cada usuario usuarios

En este TFG nos centraremos en el uso de los siguiente modelos:

Latent Dirichlet Allocation (LDA) Es un modelo probabilístico clásico que data de 2003. Se basa en el enfoque de bolsa de palabras (BoW) y asume que los documentos son una mezcla de temas y los temas una mezcla de palabras. Este es una referencia fundamental en el modelado de temas [27]. En la sección 3.4 se profundiza sobre este modelo.

Top2Vec Surge en 2020 como una alternativa que utiliza *embeddings* de palabras y documentos para encontrar temas. Este crea un espacio semántico unificado donde los vectores de temas, documentos y palabras coexisten, permitiendo una búsqueda semántica y la detección

automática del número de temas [21]. En la sección 3.4 se profundiza sobre este modelo.

BERTopic También de 2020, este modelo aprovecha el poder de los *embeddings* contextuales de los transformadores (como BERT). Su arquitectura modular combina la reducción de dimensionalidad con UMAP y el *clustering* con HDBSCAN para identificar temas semánticamente coherentes, ofreciendo una gran flexibilidad [35]. En la sección 3.4 se profundiza sobre este modelo.

FASTopic Es el modelo más reciente, de 2023, y se presenta como una evolución de los modelos basados en *embeddings*. Introduce un enfoque optimizado para ser más rápido, estable y escalable, manteniendo la alta calidad de los temas. Está diseñado para funcionar eficientemente en grandes volúmenes de datos [44]. En la sección 3.4 se profundiza sobre este modelo..

Latent Dirichlet Allocation (LDA)

LDA (del inglés, *Latent Dirichlet Allocation*, LDA) es un modelo probabilístico que usa matrices de factorización basado en la distribución de Dirichlet. En este modelo se asume que cada documento expone una serie de temas y cada tema se caracteriza por una distribución de palabras basada en la frecuencia de aparición en el documento. Los resultados obtenidos al ejecutar el modelo no son deterministas, por lo que se puede obtener un resultado distinto en cada ejecución para el mismo *dataset* [39, 26].

LDA posee dos parámetros [25] que debemos ajustar para mejorar los resultados obtenidos:

- Numero de temas
- Factor de densidad del documento-tema
- Factor de densidad tema-palabra

Top2Vec

Top2Vec es un algoritmo de modelado de temas basado en *embeddings*, documentos y vectores de palabras. Este algoritmo asume que la semántica interna de los documentos es indicativo de que existe una temática común [31].

A diferencia de otros métodos como LDA, Top2Vec genera simultáneamente *embeddings* de documentos, *embeddings* de palabras y *embeddings* de temas, lo que le permite identificar agrupaciones semánticas de forma más natural y continua. Este enfoque se basa en la idea de que los documentos que tratan temas similares deben estar próximos entre sí en un espacio vectorial de alta dimensión. Mediante el uso de modelos de lenguaje basados en redes neuronales, Top2Vec representa documentos y palabras en un espacio semántico compartido, permitiendo descubrir temas como densidades locales o agrupaciones (del inglés *clusters*) de documentos [21].

Ventajas de Top2Vec [21]:

- No requiere preprocesamiento, ya que funciona bien sin necesidad de: eliminación de *stopwords*, realizar lematización, limpiar puntuación, filtrar términos raros, etc.
- No requiere elegir el número de temas.
- Produce temas con alta coherencia semántica.
- Permite buscar documentos por similitud, identificar documentos mas representativos de un tema.

BERTopic

BERTopic es un método de modelado de temas que combina *embeddings* semánticos, reducción de dimensionalidad y *clustering* para identificar temas en grandes colecciones de texto de manera automática y precisa [35]

BERTopic se basa en el uso de BERT (del inglés, *Bidirectional Encoder Representations from Transformers*, BERT) para generar representaciones vectoriales *embeddings* de los documentos y temas. El proceso se ejecuta fundamentalmente en dos fases: la codificación semántica y la extracción de temas. En la primera fase, los documentos se transforman en *embeddings* de alta dimensionalidad mediante BERT. Posteriormente, se emplea el algoritmo HDBSCAN para agrupar estos vectores y así identificar clústeres en el espacio vectorial, permitiendo la clasificación de los textos en distintos temas. A partir de cada grupo, se derivan las palabras clave que describen el contenido [32].

BERTopic combina cuatro componentes principales:

- *Embeddings* (representaciones semánticas)

- Reducción de dimensionalidad (UMAP), una técnica que reduce las dimensiones de los *embeddings* manteniendo las relaciones importantes.
- *Clustering* basado en densidad (HDBSCAN), es un algoritmo que detecta “grupos naturales” en los datos.
- Representación final de temas (c-TF-IDF), método para identificar las palabras más representativas de cada tema.

La principal ventaja de BERTopic es su capacidad para encontrar matices semánticos y manejar grandes volúmenes de información textual. Puede identificar significados complejos y relaciones contextuales con mayor precisión. Además, a diferencia de otros métodos, no es necesario fijar de antemano cuántos temas se generarán, ya que el algoritmo de *clustering* determina este número automáticamente [32].

FASTopic

FASTopic (*Fast, Adaptative, Stable and Transferable topic model*). Es un modelo que sigue el enfoque denominado DSR (Reconstrucción de Relaciones Semánticas Dual), este no utiliza redes neuronales y se basa en tres elementos: la representación vectorial de los documentos generados por un transformador predeterminado, los vectores de temas y de palabras [44].

DSR captura dos tipos de relaciones semánticas: entre documentos y temas, y entre temas y palabras, interpretándolas como distribuciones que permiten descubrir los temas de manera ordenada y eficiente, eliminando la necesidad de estructuras complicadas. Para modelar estas relaciones, se propone además el *Embedding Transport Plan* (ETP), un mecanismo novedoso que, en lugar de emplear funciones *softmax* parametrizadas, regula estas relaciones como planes de transporte óptimo entre los *embeddings* de documentos, temas y palabras. Esto reduce el sesgo en las relaciones y permite obtener temas bien diferenciados y distribuciones más precisas [44].

FASTopic se basa en los siguientes pasos:

- Generación de *Embeddings* (representaciones semánticas).
- Reducción de dimensionalidad.
- *Clustering*, no se limitan a HDBSCAN, sino que permiten el uso de múltiples algoritmos de *clustering* que buscan eficiencia y escalabilidad.

- Extracción de palabras representativas del tema.

FASTopic tiene como principales ventajas la capacidad de manejar documentos de gran tamaño si necesidad de utilizar mayor capacidad de procesamiento, no requiere limpieza extrema del texto y produce una calidad semantica similar a la de BERTopic cuando se usan buenos *embeddings*.

3.5. Evaluación de modelos

Evaluar la calidad del resultado de los modelos es una parte esencial del proceso de análisis. Es importante verificar si los temas generados son coherentes, útiles y representativos para los textos.

La evaluación puede realizarse utilizando alguna de las siguientes técnicas:

- Utilizando métricas automáticas, que proporcionan valores numéricos objetivos.
- Evaluación humana o cualitativa, que revisa si los temas tienen sentido.
- Utilizando métricas híbridas, que combinan ambas aproximaciones.

Métricas automáticas

Las métricas permiten comparar el resultado de los modelos sin necesidad de intervención humana, las mas comunes son: perplexidad y coherencia.

Perplejidad

Es una medida utilizada en modelos probabilísticos como LDA. Indica que tan bien el modelo es capaz de predecir documentos no vistos. Cuanto mas baja mejor se considera el desempeño del modelo, implica que el modelo tiene una mayor capacidad para anticipar las palabras de textos que no han sido utilizados durante el entrenamiento.

Solo es útil para comparar versiones diferentes del mismo tipo de modelo probabilístico (LDA). No es adecuada para modelos basados en *embeddings* (BERTopic, Top2Vec, FASTopic).

Coherencia

Es una métrica que evalúa que tan relacionados están los términos de las palabras mas representativas de un tema. Esta métrica permite identificar cuantos temas son adecuados para un conjunto de datos determinado [40].

La coherencia busca medir hasta qué punto las palabras más representativas de un tópico tienden a aparecer juntas en el corpus y a mantener relaciones semánticas coherentes. Una mayor coherencia indica un tópico más interpretable y semánticamente consistente.

Los métodos más usados son:

- **C_V**: basada en similitud semántica.
- **UCI**: mide coocurrencias de palabras en ventanas deslizantes.
- **UMass**: calcula coocurrencias dentro del propio corpus.

Esta métrica funciona bien tanto en LDA como en BERTopic, Top2Vec o FASTopic.

Evaluación Humana

El investigador revisa la lista de términos que componen cada tema y determina si son coherentes. En investigación cualitativa es habitual que varias personas evalúen los `phdTopicModelingAlgorithms2023` y se calcule el índice de acuerdo inter-evaluador. Esto ayuda a evitar que la evaluación dependa de la subjetividad de una sola persona [43]

3.6. Hiperparametrización de Modelos

Los hiperparámetros son valores configurados por el investigador antes de entrenar un modelo. Determinan el comportamiento del algoritmo, la calidad de los temas obtenidos y el tiempo de cómputo. Ajustarlos adecuadamente es esencial para obtener resultados óptimos en el análisis temático [27, 34].

Hiperparámetros en LDA

LDA es uno de los modelos más sensibles a su configuración.

Número de temas (k)

Controla cuántos temas debe encontrar el modelo. Valores demasiado bajos generan temas demasiado generales, mientras que valores demasiado altos producen temas ruidosos o redundantes.

Parámetro α

Regula cuántos temas aparecen en cada documento. Valores altos implican documentos con muchos temas mezclados; valores bajos generan documentos dominados por pocos temas.

Parámetro β

Determina cuántas palabras componen cada tópico. Un valor alto genera temas más dispersos y un valor bajo produce temas más específicos.

Número de Iteraciones

Afecta la estabilidad del modelo. Más iteraciones proporcionan mejor convergencia a costa de un mayor tiempo de entrenamiento.

Hiperparámetros en BERTopic

BERTopic tiene hiperparámetros en varias etapas: embeddings, reducción dimensional (UMAP), clustering (HDBSCAN) y extracción de palabras mediante c-TF-IDF.

UMAP

Los hiperparámetros más relevantes son:

- **n_neighbors**: controla la influencia del vecindario local. Valores bajos encuentran temas más específicos.
- **n_components**: número de dimensiones finales.

HDBSCAN

Los ajustes más influyentes son:

- **min_cluster_size**: tamaño mínimo del grupo.
- **min_samples**: sensibilidad al ruido.

Hiperparámetros en Top2Vec

Top2Vec utiliza hiperparámetros principalmente en el modelo de embeddings (Doc2Vec, BERT o USE) y en configuraciones de UMAP y HDBSCAN, compartiendo muchos de los ajustes de BERTopic.

Hiperparámetros en Fastopic

Fastopic está optimizado para grandes volúmenes de datos y permite configurar:

- tipo de embeddings (BERT, FastText, etc.).
- número de clusters en métodos rápidos como K-Means.
- número de palabras clave extraídas por tópico.

Tanto la evaluación como la hiperparametrización de modelos de temas son procesos fundamentales para garantizar la calidad del análisis temático. Modelos como LDA requieren ajustes cuidadosos en parámetros probabilísticos, mientras que modelos modernos como BERTopic, Top2Vec y Fastopic dependen más de configuraciones de embeddings, reducción de dimensionalidad y clustering. La combinación de métricas automáticas y evaluación humana proporciona la estrategia más robusta para obtener temas coherentes y útiles en contextos como el análisis de TFG.

Métodos de búsqueda de hiperparámetros

Existen diversas estrategias para ajustar hiperparámetros, con distintos niveles de complejidad y eficiencia:

- **Grid Search:** explora todas las combinaciones posibles dentro de un conjunto definido. Sin embargo, resulta poco eficiente cuando el número de hiperparámetros es elevado [30].
- **Random Search:** selecciona combinaciones aleatorias de hiperparámetros dentro de los rangos definidos. Ha demostrado ser más eficiente que la búsqueda exhaustiva en espacios de gran dimensionalidad [30].
- **Optimización Bayesiana:** construye un modelo probabilístico del rendimiento y selecciona nuevas combinaciones basadas en regiones prometedoras del espacio de búsqueda. Es una opción muy eficiente cuando las evaluaciones del modelo son costosas [42].

3.7. Comparación de modelos

Con el objetivo de visualizar las diferencias de los distintos modelos seleccionados, en la Tabla 3.1 se presentan las propiedades de los cuatro algoritmos de modelado de temas evaluados. Esta comparación permite visualizar la evolución desde los modelos probabilísticos clásicos hacia los más recientes.

Características	LDA	Top2Vec	BERTopic	FASTopic
Requiere indicar numero de temas k	Sí	No	No	No
Uso de <i>embeddings</i>	No	Sí	Sí	Sí
Representación del texto	BOW	<i>embeddings</i> semánticos	<i>embeddings</i> contextuales	<i>embeddings</i> contextuales
Preprocesamiento necesario	Alto	Muy bajo	Moderado	Bajo
Coherencia de temas	Media	Muy Alta	Alta	Muy Alta
Coste computacional	Bajo	Alto	Alto	Alto
Interpretabilidad	Alta	Media	Media-Alta	Media

Tabla 3.1: Comparativa de las características de los algoritmos de modelado de temas.

4. Técnicas y herramientas

En este capítulo se describen las técnicas y herramientas utilizadas durante el desarrollo del proyecto. Se abordan tanto las metodologías empleadas como las herramientas de software que facilitaron la implementación y gestión del mismo.

4.1. Metodologías de desarrollo

El desarrollo del proyecto se llevó a cabo utilizando la metodología ágil **SCRUM** [20] de forma adaptada al contexto de un proyecto académico individual. Esta metodología permitió una gestión flexible y adaptativa del proyecto. El trabajo se organizó en *sprints* de dos semanas, donde se definieron objetivos claros y se realizaron revisiones periódicas para evaluar el progreso. Frente a metodologías estrictamente secuenciales, este enfoque resulta especialmente adecuado en proyectos de carácter experimental, como es el caso del modelado de temas, donde los resultados dependen en gran medida de la calidad del preprocesado y de la configuración de los modelos.

Para la planificación de tareas y realizar seguimiento del proyecto se utilizó la aplicación web **Zube** [46]. Esta herramienta se integra fácilmente con GitHub, permite crear *sprints*, épicas y tableros Kanban.

Para seguir el flujo de trabajo de SCRUM y facilitar el proceso de desarrollo, en este caso el tutor adoptó los roles de product owner y SCRUM master.

4.2. Herramientas de gestión de proyectos

Para la gestión del proyecto se utilizó la plataforma **GitHub**. Esta herramienta permitió el alojamiento y gestión del código fuente, permitiendo llevar un control de versiones eficiente utilizando Git [3].

4.3. Herramientas de desarrollo

Entorno de desarrollo

El entorno de desarrollo utilizado fue **Visual Studio Code**, un editor de código fuente ligero y gratuito desarrollado por Microsoft [12]. Este entorno ofrece una amplia gama de extensiones y funcionalidades que facilitan la escritura, depuración y gestión de código en múltiples lenguajes de programación. Fue una herramienta esencial para el desarrollo de este proyecto gracias a su amplia integración con *plugins* que ayudan al desarrollo y depuración con el lenguaje de programación Python. Además, Visual Studio Code cuenta con integración nativa con GitHub, lo que facilita la gestión de versiones. Se valoró el uso de otros entornos como **PyCharm** [19], pero se optó por Visual Studio por su ligereza y flexibilidad.

También se utilizó **Jupyter Notebooks** [9] para la experimentación y análisis de datos, ya que permite combinar código, visualizaciones y documentación en un solo documento interactivo.

Lenguajes de programación

El lenguaje de programación principal utilizado en el desarrollo del proyecto fue **Python** [10]. Python es un lenguaje de alto nivel, interpretado y de propósito general, conocido por su sintaxis clara. Su elección se basa en la amplia gama de bibliotecas y *frameworks* para el procesamiento de datos y modelado de temas disponibles.

Bibliotecas y frameworks

El núcleo del proyecto se apoya en técnicas de NLP aplicadas al análisis de documentos. Para el procesamiento de datos y modelado de temas se utilizaron diversas bibliotecas de Python, entre las que destacan:

- **Pandas** [16]: Biblioteca fundamental para la manipulación y análisis de datos.

- **NumPy** [7]: Biblioteca para computación científica y manejo de arreglos multidimensionales.
- **Gensim** [2]: Biblioteca especializada en modelado de temas y procesamiento de lenguaje natural.
- **NLTK** [6]: Biblioteca para el procesamiento de lenguaje natural, usado para tareas básicas tokenización y *stopwords*.
- **spaCy** [14]: Biblioteca avanzada para procesamiento mas avanzado de lenguaje natural.

Sobre los textos preprocesados se aplican técnicas de modelado de temas. Se han estudiado y utilizado distintos enfoques, desde modelos probabilísticos clásicos hasta métodos más recientes basados en representaciones vectoriales. La comparación entre estas técnicas permite analizar sus fortalezas y limitaciones en función de criterios como la coherencia semántica de los temas, la interpretabilidad y el coste computacional.

Se utilizaron las siguientes bibliotecas de Python para el modelado de temas:

- **LDA** [5]: Implementación del modelo Latent Dirichlet Allocation para descubrir temas en grandes colecciones de documentos.
- **BERTopic** [13]: Biblioteca que utiliza modelos de lenguaje basados en transformadores para generar representaciones de documentos y realizar clustering.
- **Top2Vec** [22]: Biblioteca que combina incrustaciones de documentos y clustering para identificar temas en grandes conjuntos de datos.
- **FASTopic** [1]: Biblioteca que utiliza técnicas de reducción de dimensionalidad y clustering para el modelado eficiente de temas en grandes conjuntos de datos.

En cuanto a la experimentación con modelos y la optimización de hiperparámetros, se ha utilizado **Optuna** [8] como herramienta principal para la exploración del espacio de configuraciones. Esta biblioteca permite definir procesos de búsqueda automatizados y eficientes y facilita la comparación objetiva de los resultados mediante métricas cuantitativas como la coherencia de los temas. Frente a la selección manual de parámetros, este enfoque

resulta más riguroso y contribuye a la reproducibilidad y trazabilidad de los experimentos realizados.

Finalmente, para la presentación y exploración de los resultados se ha desarrollado un interfaz web utilizando la librería **Streamlit** [15]. Su elección se debe principalmente a que esta herramienta permite construir interfaces web interactivas de forma ágil sin entrar en detalles complejos de desarrollo web. Streamlit facilita la creación de paneles donde se pueden cargar datos y ajustar parámetros facilitando la visualización de los modelos generados y de sus métricas asociadas.

Otras herramientas

Además de las herramientas mencionadas anteriormente, se utilizaron diversas herramientas de apoyo orientadas a la gestión del entorno de ejecución y a la calidad del software desarrollado.

En primer lugar, se ha utilizado **Docker** [17] y **Docker Compose** como tecnología de contenedorización. Docker permite encapsular la aplicación junto con todas sus dependencias en entornos reproducibles y aislados, lo que simplifica tanto el despliegue como la ejecución del sistema en diferentes máquinas. Este enfoque resulta especialmente útil ya que reduce problemas derivados de incompatibilidades de versiones y facilita la portabilidad y reproducibilidad de los experimentos.

Por otro lado, se han utilizado sistemas orientados a garantizar la calidad del código. En primer lugar, para el formateo automático del código se ha utilizado **Black**, lo que asegura un estilo de código uniforme en todo el proyecto. La organización y limpieza de importaciones se ha gestionado usando **isort**, mientras que el análisis estático y la detección de posibles errores o incumplimientos de estilo se han realizado con **flake8**. Estas herramientas mejoran la legibilidad del código y reducen la aparición de errores durante el desarrollo.

4.4. Documentación

Para la creación de la documentación del proyecto se utilizó **LaTeX** [4]. LaTeX es especialmente adecuado para la creación de documentos técnicos y científicos debido a su capacidad para manejar fórmulas matemáticas complejas, referencias bibliográficas y estructuras avanzadas de documentos. Además, LaTeX permite separar el contenido de la presentación, facilitando la gestión y el formato del documento final. Se optó por LaTeX frente a

procesadores de texto tradicionales como Microsoft Word o Google Docs, ya que estos últimos pueden presentar limitaciones en la gestión de referencias y fórmulas complejas. Además, la universidad proporciona plantillas pre formateadas y estandarizadas para la creación de la memoria en LaTeX. Como editor de LaTeX se utilizó Visual Studio Code con la extensión **LaTeX Workshop** [45].

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros³, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

5.1. Arquitectura y diseño del software

Para el diseño del código se empleó una arquitectura de tipo hexagonal [18]. Este enfoque permite desacoplar la lógica de negocio de los detalles de infraestructura, como el acceso a datos, la ejecución de modelos o la interfaz de usuario. La principal ventaja de esta arquitectura frente a diseños más monolíticos es que reduce el impacto de cambios tecnológicos, aspecto rele-

vante en un proyecto que combina procesamiento de datos, experimentación con modelos y visualización de resultados.

Aunque existen alternativas como arquitecturas en capas tradicionales o enfoques basados en microservicios, la arquitectura hexagonal ofrece un equilibrio adecuado entre simplicidad conceptual y robustez, siendo especialmente apropiada para un proyecto académico de tamaño medio.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Bibliografía

- [1] Fastopic: FASTopic. <https://github.com/bobxwu/fastopic>.
- [2] Gensim: Topic modelling for humans. https://radimrehurek.com/gensim/auto_examples/index.html#documentation.
- [3] Git - Reference. <https://git-scm.com/docs>.
- [4] LaTeX - A document preparation system. <https://www.latex-project.org/>.
- [5] Lda: Topic modeling with latent Dirichlet Allocation — lda documentation. <https://lda.readthedocs.io/en/latest/>.
- [6] NLTK :: Natural Language Toolkit. <https://www.nltk.org/>.
- [7] NumPy user guide — NumPy v1.26 Manual. <https://numpy.org/doc/1.26/user/index.html#user>.
- [8] Optuna - A hyperparameter optimization framework. <https://optuna.org/>.
- [9] Project Jupyter Documentation — Jupyter Documentation 4.1.1 alpha documentation. <https://docs.jupyter.org/en/latest/>.
- [10] Python 3.10.19 Documentation. <https://docs.python.org/3.10/>.
- [11] ¿Qué es la tokenización? Tipos, casos de uso, aplicación. <https://www.datacamp.com/blog/what-is-tokenization>.
- [12] ¿Qué es Visual Studio Code y cuáles son sus ventajas? | Blog de Arsys | Blog de Arsys. <https://www.arsys.es/blog/que-es-visual-studio-code-y-cuales-son-sus-ventajas>.

- [13] Quick Start - BERTopic. https://maartengr.github.io/BERTopic/getting_started/quickstart/quickstart.html.
- [14] spaCy 101: Everything you need to know · spaCy Usage Documentation. <https://spacy.io/usage/spacy-101>.
- [15] Streamlit Docs. <https://docs.streamlit.io/>.
- [16] User Guide — pandas 2.3.3 documentation. https://pandas.pydata.org/docs/user_guide/index.html.
- [17] Docker manuals. <https://docs.docker.com/manuals/>, 09:11:23 +0100 +0100.
- [18] Hexagonal Architecture and Clean Architecture (with examples). <https://dev.to/dyarleniber/hexagonal-architecture-and-clean-architecture-with-examples-48oi>, 2022.
- [19] PyCharm : Todo sobre el IDE de Python más popular. <https://datascientest.com/es/pycharm>, 2022.
- [20] Agile Alliance. Agile 101 - What is Agile? <https://www.agilealliance.org/agile101/>, 2024. [En línea; consultado el 19-septiembre-2024].
- [21] Dima Angelov. Top2Vec: Distributed Representations of Topics, 2020. <http://arxiv.org/abs/2008.09470>.
- [22] Dima Angelov. Ddangelov/Top2Vec. <https://github.com/ddangelov/Top2Vec>, 2025.
- [23] John Atkinson-Abutridy. *Analítica textual*. Marcombo.
- [24] John Atkinson-Abutridy. *Grandes modelos de lenguaje. Conceptos, técnicas y aplicaciones*. Marcombo, 2023.
- [25] Aishwarya Bhangale. Introduction to Topic Modelling with LDA, NMF, Top2Vec and BERTopic, 2023. <https://medium.com/blend360/introduction-to-topic-modelling-with-lda-nmf-top2vec-and-bertopic-ffc3624d44e4>, [Internet, descargado 10-noviembre-2025].
- [26] Iqra Bismi. Topic modelling using LDA, 2023. <https://medium.com/@iqra.bismi/topic-modelling-using-lda-fe81a2a806e0>, [Internet, descargado 10-noviembre-2025].

- [27] David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.
- [28] Castillo-Velásquez Francisco Antonio Campos-Sánchez Jonathan Josefatz, Gonzalez-Gutierrez Fidel. Revisión Sistemática de Técnicas de Tokenización para la Clasificación de Información en Inteligencia Artificial. https://virtual.cuautitlan.unam.mx/intar/memoriasceiaait/wp-content/uploads/sites/19/2024/12/24-Revision-Sistemática-de-Técnicas-de-Tokenización-__para-la-Clasificación-de-Información-en-Inteligencia-Artificial-EDITADO.pdf.
- [29] Gustavo Espíndola. ¿Qué son los embeddings y cómo se utilizan en la inteligencia artificial con python?, 2023. <https://gustavo-espindola.medium.com/qu%C3%A9-son-los-embeddings-y-c%C3%B3mo-se-utilizan-en-la-inteligencia-artificial-con-python-45b751ed86a5>.
- [30] Luca Franceschi, Michele Donini, Valerio Perrone, Aaron Klein, Cédric Archambeau, Matthias Seeger, Massimiliano Pontil, and Paolo Frasconi. Hyperparameter optimization in machine learning. *Foundations and Trends® in Machine Learning*, 18(6):1054–1201, 2025. <http://dx.doi.org/10.1561/22000000088>.
- [31] Thiyaneshwaran G. Top2vec for Topic Modeling and Semantic Similarity and Search, 2022. <https://medium.com/@ThiyaneshwaranG/top2vec-for-topic-modeling-and-semantic-similarity-and-search-77db82dda7d3>.
- [32] Lin Gan, Tao Yang, Yifan Huang, Boxiong Yang, Yami Yanwen Luo, Lui Wing Cheung Richard, and Dabo Guo. Experimental comparison of three topic modeling methods with lda, top2vec and bertopic. In *International Symposium on Artificial Intelligence and Robotics*, pages 376–391. Springer, 2023.
- [33] gewarren. Descripción de los tokens - .NET. <https://learn.microsoft.com/es-es/dotnet/ai/conceptual/understanding-tokens>.
- [34] Thomas L Griffiths and Mark Steyvers. Finding scientific topics. *PNAS*, 101(suppl 1):5228–5235, 2004.
- [35] Maarten Grootendorst. BERTopic: Neural topic modeling with a class-based TF-IDF procedure, 2022. <http://arxiv.org/abs/2203.05794>.

- [36] Mustapha Hankar, Mohammed Kasri, and Abderrahim Beni-Hssane. A comprehensive overview of topic modeling: Techniques, applications and challenges. *Neurocomputing*, 628:129638, 2025. <https://linkinghub.elsevier.com/retrieve/pii/S0925231225003108>.
- [37] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2025. <https://web.stanford.edu/~jurafsky/slp3/>.
- [38] Carlos López Nozal, Raúl Marticorena Sánchez, Juan José Rodríguez, and Andrés Bustillo Iglesias. Proceso de gestión de trabajos fin de carrera. In *Jornadas de Enseñanza Universitaria de la Informática*, pages 413–420. Jornadas de Enseñanza Universitaria de la Informática, 2009.
- [39] Madhurima Nath. Topic modeling algorithms, 2023. <https://medium.com/@m.nath/topic-modeling-algorithms-b7f97cec6005>, [Internet; descargado 10-noviembre-2025].
- [40] João Pedro. Understanding Topic Coherence Measures. <https://towardsdatascience.com/understanding-topic-coherence-measures-4aa41339634c/>, 2022.
- [41] Serhad Sarica and Jianxi Luo. Stopwords in technical language processing. *Plos one*, 16(8), 2021. <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0254937>.
- [42] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical bayesian optimization of machine learning algorithms, 2012. <https://arxiv.org/abs/1206.2944>.
- [43] Dr Tiya Vaj. How to evaluate novel topic modeling method. <https://vtiya.medium.com/how-to-evaluate-novel-topic-modeling-method-104ad9684428>, 2023.
- [44] Xiaobao Wu, Thong Nguyen, Delvin Ce Zhang, William Yang Wang, and Anh Tuan Luu. FASTopic: Pretrained Transformer is a Fast, Adaptive, Stable, and Transferable Topic Model. *arXiv e-prints*, 2024.
- [45] James Jianqiao Yu. James-Yu/LaTeX-Workshop. <https://github.com/James-Yu/LaTeX-Workshop>, 2025.
- [46] Zube. Zube - project management for agile development. <https://zube.io>. Accessed: 2024-09-19.